

Chagas Disease Detection from ECG Data

Jonathan Maenche

GitHub code: <https://github.com/MeanGold/4300-data-science-project>

Introduction

My project will be based on the George B. Moody PhysioNet Challenge 2025¹. This challenge is aimed at detecting Chagas disease from data in electrocardiograms (ECGs). Chagas disease is a parasitic disease in Central and South America. It affects an estimated 6.5 million people and causes nearly 10,000 deaths annually². Testing via blood samples is possible in some places, and this testing (serological testing) is successful at detecting Chagas disease. Serological testing, however, is not readily available in Central and South America, which is where the disease is primarily found. Chagas symptoms may also show up in patient ECG data, so the challenge is to develop a project that is able to predict Chagas disease based on patient ECGs. The predictions based on ECG results can then be used to prioritize serological testing for those individuals with a higher Chagas likelihood based on their ECG data. Detecting Chagas earlier through these means could translate to better treatment and less risk of death for patients³.

I will be using a binary classification algorithm using two approaches: a logistic regression model aimed at using discrete variables to predict the probability of Chagas disease for a patient, and a convolutional neural network approach that attempts to tune hidden model parameters to best predict the probability of Chagas disease for a patient.

This algorithm could then be used to screen patients before serological testing is conducted, and patients that are identified as potential positive candidates for Chagas could undergo serological testing to determine if they have the disease.

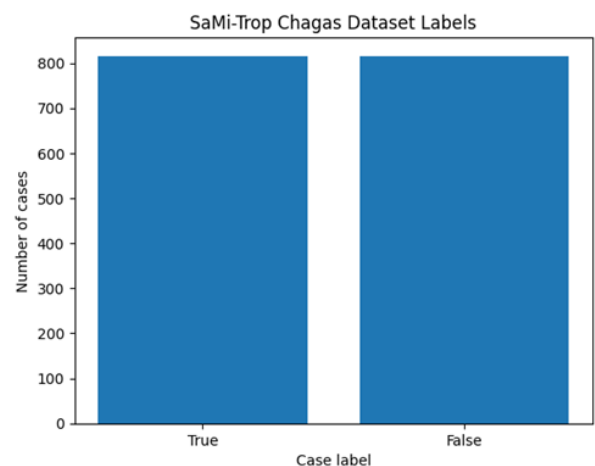
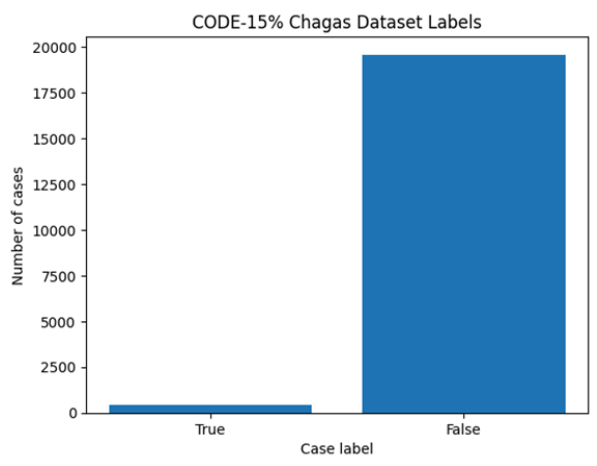
Methods

Data source and description

The PhysioNet Challenge had lots of data that could be used for developing a model. To simplify my approach and reduce the overall amount of data that I had to work with, I chose to use the SaMi-Trop⁴ dataset and a subset of the CODE-15%⁵ dataset.

The SaMi-Trop dataset was gathered with the explicit purpose of discovering if there is a link between patient ECG data and Chagas disease. It consists of 1631 ECG exams, approximately half of which are positive for Chagas. I utilized all the exams when developing my model.

The CODE-15% dataset contains 345,779 ECG exams and is a subset of an even larger dataset. Due to the large size of this dataset, I only utilized a portion of the dataset for my model. The portion that I used contains 20,000 ECG exams. This dataset contains a very small percentage of exams that are positive for Chagas (approximately 2.08%).

SaMi-Trop dataset	CODE-15% (part1) dataset
Number of positive cases: 815 Number of negative cases: 816	Number of positive cases: 417 Number of negative cases: 19584
 SaMi-Trop Chagas Dataset Labels	 CODE-15% Chagas Dataset Labels

Data preprocessing

The data in SaMi-Trop and the CODE-15% datasets are stored with the ECG exams and the labels in separate data structures, so one of my primary problems was trying to link the two. To solve this, I took the exam ID and identified it in a list containing the ECG exam ID and a Chagas label.

I only had to conduct feature extraction for my logistic regression model (baseline model). This required analyzing each ECG exam in my data using a software module I found online for ECG statistics. Unfortunately, the module also introduced errors into these features due to certain properties in individual ECG exams, so I then had to filter out some of the features and drop some rows that contained additional errors. Though this marginally reduced the size of my dataset, I strove to keep as many samples as possible. Once I had the features extracted and cleaned, I was able to run my baseline model and compile some results from it.

No feature extraction was performed to produce my model based on a convolutional neural network.

Baseline analysis / model

I ran an initial logistic regression model using features collected on all 12 ECG leads such as average heart rate and average R-to-R peak distance. I also included features for individual leads such as the standard deviation for the heart rate, the variance of the R-to-R peak distance, and

the length from the start of the QRS complex to the T-wave. I set up my model to use several hyperparameters. These are listed below...

Hyperparameters	Options
C (inverse of regularization)	0.0001, 0.001, 0.01, 0.1, 1, 10
Penalty	L1, L2, None
Solver	saga
SMOTE oversampling threshold	0.1, 0.25, 0.5, 0.75, 1.0

As can be seen from the last hyperparameter, I utilized oversampling to try to address the class imbalance problem in my dataset. I tested both manipulating class weights and the Synthetic Minority Over-sampling Technique (SMOTE), and SMOTE seemed to achieve a better class balance with model predictions.

Alternative analysis / model

For my alternative model, I built a simple Convolutional Neural Network for analyzing each of the 12 leads in a patient's ECG exam. I used a single convolutional layer that took in the 4096 signals for each of the 12 leads and ran them through a kernel. Once I had the encodings, I flattened them to a single linear layer, and then condensed those into a binary class prediction.

Training and evaluation paradigm

For my baseline model, I used grid search with 5-fold cross validation to find the best hyperparameters combinations. I also used the *train_test_split* function to randomly shuffle and split the data into 75% training and 25% test subsets.

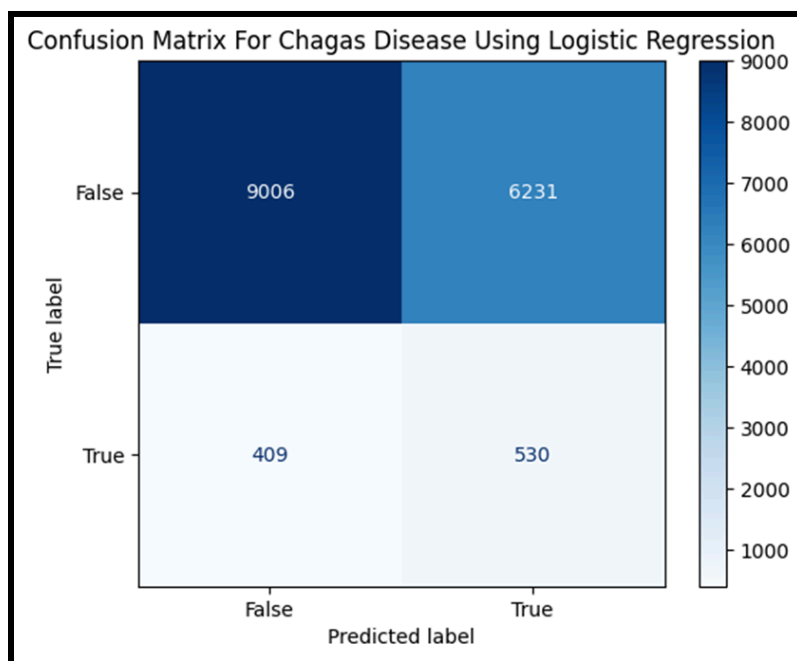
For my alternative model, I split my data into 70% training, 20% validation, and 10% test subsets. I used the *train_test_split* function to randomly shuffle and split the data into these subsets. In addition, my neural network used binary cross entropy as the loss function to adjust model parameters during training and validation. This loss function is designed to help solve binary classification problems and updates the weights within the neural network.

For both models, I chose to use precision, recall, and f1-score to evaluate my model. I will produce a classification report with these metrics and a confusion matrix showing the true positive, true negative, false positive, and false negative values.

Results

Baseline

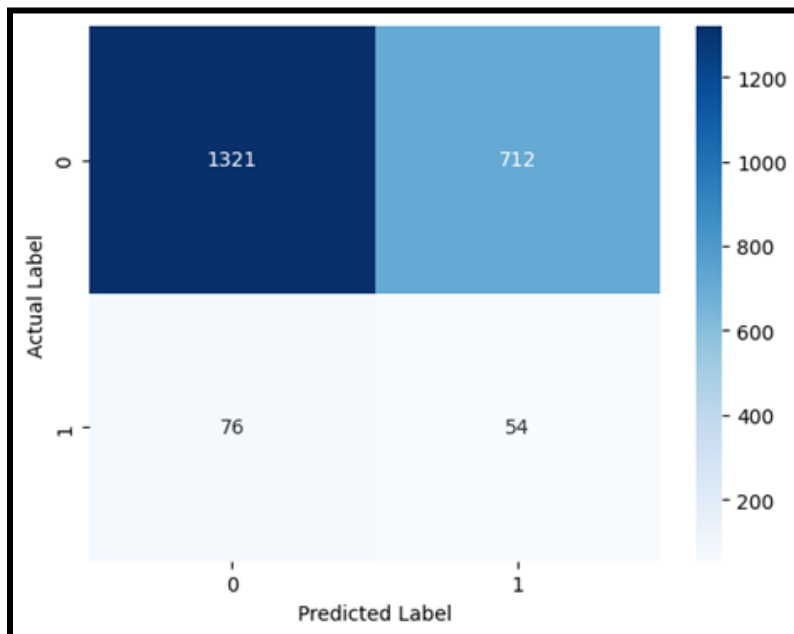
<u>CLASSIFICATION REPORT</u>				
	Precision	Recall	F1	Support
<i>False</i>	0.96	0.59	0.73	15237
<i>True</i>	0.08	0.56	0.14	939
Accuracy			0.59	16176
Macro avg	0.52	0.58	0.43	16176
Weighted avg	0.91	0.59	0.70	16176



My baseline model does not perform very well. My goal in training was to try to get as many values into the true positive and true negative categories. As you can see for each of the actual labels, it has a 0.56 recall for the positive Chagas cases and 0.59 recall for the negative Chagas cases. In this case, my model is heavily overpredicting the positive Chagas label in order to try to increase the model's ability to find all positive cases. This sacrifices precision for the positive Chagas label, and the model produces a lot of false positives. Therefore, this model does a relatively poor job of distinguishing the two classes.

Alternative

<u>CLASSIFICATION REPORT</u>				
	Precision	Recall	F1-score	Support
<i>False</i>	0.9456	0.6498	0.7703	2033
<i>True</i>	0.0705	0.4154	0.1205	130
Accuracy			0.6357	2163
Macro avg	0.5080	0.5326	0.4454	2163
Weighted avg	0.8930	0.6357	0.7312	2163



My alternative model also does not perform well on this class imbalance problem. In order to try to address the class imbalance, I originally set out to use a weighted random sampler to try and address this problem, but that did not work and seemed to only draw samples from the negative class. This made all my training samples negative and resulted in the model only producing negative predictions. This is not what I wanted, so instead I added the weight for the positive class to the loss function within the training step of my convolutional neural network (CNN). This resulted in a better balance of predictions as seen above. Like my original baseline mode, my alternative model with a CNN had difficulty distinguishing between the positive and negative Chagas cases. The alternative model similarly produced many false positives. In contrast to the baseline model, the CNN had a lower recall for the positive class, meaning that it correctly

predicted a lower percentage of the positive class. Thus, my alternative model performed slightly worse than the baseline model.

Discussion & Conclusion

My baseline model was relatively poor at distinguishing between the two classes. This may be due to the limitations of a linear regression model on a dataset with an imbalanced class. Thus, I would not recommend using this type of model for actual Chagas detection. With further improvement and research into key ECG features for Chagas disease, a linear regression model could achieve better performance and maybe be utilized to provide initial screening for Chagas. Patients selected by the model as at risk for Chagas could then undergo serological testing to determine if they actually have the disease. Unfortunately, the current model performance is not good enough for it to truly bring an advantage to this specific problem.

My alternative model also had relatively poor performance. My neural network is fairly simple, and this likely impacted my model performance. My network includes a single convolutional layer, a pooling layer followed by a ReLU layer. These values were then flattened and passed to the linear layer to produce a single output prediction. My model might benefit from an additional one or two convolutional layers, but the current model does not perform well at predicting Chagas disease based on ECG data. Therefore, I would not recommend using this model for actual Chagas detection.

Overall, the Chagas detection problem using ECG data has proven to be a very challenging problem to solve. I have learned about the difficulties of training models on problems with high class imbalance. I worked to address the problems with class imbalance in my dataset using different implementations of SMOTE and applying class weights. Though these helped to keep the model from predicting only the dominant class, neither of these options seemed to provide a true advantage in terms of model performance. In conclusion, I would not recommend using either model for actual Chagas detection, but I believe that these models could be altered and improved with further research and development.

Additional model refinement & work

As another avenue for improving model performance, I decided to collect the features for all 12 leads in the ECG as my next step since my current logistic regression model only had 4 features. When collecting the features for all 12 leads, I decided to calculate the *overall* average for two ECG features... average heart rate for each lead, and average R to R distance for each lead. I looked at the data for several samples and determined that these were pretty much the same across all 12 leads. Thus, rather than produce the average value for every single lead, I decided to combine each of these into a single feature – average heart rate of all 12 leads and average R to R distance of all 12 leads.

After running feature extraction for all 12 leads, I realized that the feature extraction process had created null or empty values in my dataset for several variables. I decided to eliminate certain features that had a high number of null or empty values. These included...

- Standard deviation of the heart rate, R to R variance, and average Q-T length for lead 4
- Standard deviation of the heart rate, R to R variance, and average Q-T length for lead 7
- Average Q-T length for leads 9, 10, 11, and 12

By eliminating these features, I was able to reduce the number of rows with null values that then had to be eliminated from the dataset for model fitting.

Dataset name	# of rows with null/empty values	% of total dataset
<i>Before feature reduction...</i>		
SaMi-Trop	374	22.9%
CODE-15%	3374	16.9%
<i>After feature reduction...</i>		
SaMi-Trop	156	9.6%
CODE-15%	966	4.8%

Analysis

Adding more features did not seem to provide much benefit to the model performance. It did provide a nice challenge for data wrangling and data preprocessing.

SOURCES

Link to GitHub with code... <https://github.com/MeanGold/4300-data-science-project>

- 1 – “Detection of Chagas Disease from the ECG: The George B. Moody PhysioNet Challenge 2025.” *George B. Moody Physionet Challenge*, moody-challenge.physionet.org/2025. Accessed 1 May 2025.
- 2 – Cucunubá, Z. M., Gutiérrez-Romero, S. A., Ramírez, J., Velásquez-Ortiz, N., Ceccarelli, S., Parra-Henao, G., Henao-Martínez, A. F., Rabinovich, J., Basáñez, M., Nouvellet, P., & Abad-Franch, F. (2024). The epidemiology of Chagas disease in the Americas. *The Lancet Regional Health - Americas*, 37, 100881. doi: 10.1016/j.lana.2024.100881
- 3 – Jidling C, Gedon D, Schön TB, Oliveira CDL, Cardoso CS, Ferreira AM, Giatti L, Barreto SM, Sabino EC, Ribeiro ALP, Ribeiro AH. Screening for Chagas disease from the electrocardiogram using a deep neural network. *PLoS Negl Trop Dis*. 2023 Jul 3;17(7):e0011118. doi: 10.1371/journal.pntd.0011118. PMID: 37399207; PMCID: PMC10361500.
- 4 – Cardoso CS, Sabino EC, Oliveira CD, de Oliveira LC, Ferreira AM, Cunha-Neto E, Bierrenbach AL, Ferreira JE, Haikal DS, Reingold AL, Ribeiro AL. Longitudinal study of patients with chronic Chagas cardiomyopathy in Brazil (SaMi-Trop project): a cohort profile. *BMJ Open*. 2016 May 4;6(5):e011181. doi: 10.1136/bmjopen-2016-011181. PMID: 27147390; PMCID: PMC4861110.
- 5 – Ribeiro AH, Ribeiro MH, Paixão GMM, Oliveira DM, Gomes PR, Canazart JA, Ferreira MPS, Andersson CR, Macfarlane PW, Meira W Jr, Schön TB, Ribeiro ALP. Automatic diagnosis of the 12-lead ECG using a deep neural network. *Nat Commun*. 2020 Apr 9;11(1):1760. doi: 10.1038/s41467-020-15432-4. Erratum in: *Nat Commun*. 2020 May 1;11(1):2227. doi: <https://doi.org/10.1038/s41467-020-16172-1>. PMID: 32273514; PMCID: PMC7145824.